EXERCISE 2

# Website Vulnerability Scanning Report

Lab: Port Scanning with Pen-test Tools

| | |
|---|---|
| **Platform** | Pentest-Tools.com — Website Vulnerability Scanner |
| **Target** | Temporary Google Gruyere training instance |
| **Date performed** | 10 October 2026 |
| **Scan profile** | Light • Authentication disabled |
| **Status** | Completed |

**Scope and authorization:** This exercise followed the supplied training lab and used a temporary Google Gruyere instance created for the exercise. Google Gruyere is intentionally vulnerable and warns users not to submit personal or private data. This report documents scanner observations; it does not claim exploitation or a full security assessment.

## 1. Objective

Run a light website vulnerability scan against the lab-provided Google Gruyere instance, review the resulting findings, understand the scanner evidence and risk ratings, and preserve the results for a technical portfolio.

## 2. Scan Configuration and Statistics

| Setting | Recorded value |
|---|---|
| Scan type | Light |
| Authentication | Disabled |
| Custom scan time | Off |
| Redirect handling | Same-domain only |
| Scan duration | 17 seconds |
| Tests performed | 40 |
| Unique injection points | 11 |
| URLs spidered | 11 |
| HTTP requests | 22 |
| Average response time | 194 ms |

## 3. Executive Summary

The scanner reported an **overall risk level of Medium**, with 13 findings in total. These are results for this particular training instance and scan configuration, not a certification that the application is secure.

| Risk category | Count |
|---|---|
| Critical | 0 |
| High | 0 |
| Medium | 2 |
| Low | 7 |
| Informational | 4 |
| Total | 13 |

Important coverage limitation: the original report explicitly states that the Light scan did not check critical issue classes such as SQL injection (SQLi), cross-site scripting (XSS), command injection, and XML external entity (XXE) vulnerabilities.

# 4. Findings Observed

| # | Finding | Scanner label | Evidence / interpretation |
|---|---|---|---|
| 1 | Insecure cookie setting: missing Secure flag | Confirmed | GRUYERE cookie response did not specify Secure. |
| 2 | Insecure cookie setting: missing HttpOnly flag | Confirmed | GRUYERE cookie response did not specify HttpOnly. |
| 3 | Missing X-Content-Type-Options header | Confirmed | Response did not include this header. |
| 4 | Missing Content-Security-Policy header | Confirmed | Scanner did not find a CSP header or meta policy. |
| 5 | Missing Referrer-Policy header | Confirmed | Header and corresponding meta tag were not found. |
| 6 | Missing Strict-Transport-Security header | Confirmed | Response did not include HSTS. |
| 7 | robots.txt file found | Confirmed | Publicly accessible file; entries are not access controls. |
| 8 | Password submitted in URL | Unconfirmed | Forms using GET with password inputs were identified. |
| 9 | Server software and technology found | Unconfirmed | Google Cloud-related components and HTTP/3 were fingerprinted. |
| 10 | Login interface found | Confirmed | Login/account-creation interface detected; this alone does not prove brute-force exposure. |
| 11 | File upload | Confirmed | Upload interface detected; unsafe handling was not proven by its presence alone. |
| 12 | security.txt file is missing | Confirmed | Scanner reported missing /.well-known/security.txt. |
| 13 | HTTP OPTIONS enabled | Confirmed | Recorded response was HTTP 405 with Allow: GET, POST; interpret carefully. |

The labels Confirmed and Unconfirmed preserve the scanner report wording. A detected interface or feature does not automatically mean it is exploitable. The OPTIONS result deserves careful review because the recorded response is 405 Method Not Allowed with Allow: GET, POST.

# 5. Recommended Remediation

• **Cookie attributes:** Set Secure for cookies that should only be sent over HTTPS and HttpOnly for session cookies that do not need JavaScript access. Review SameSite as part of session-cookie configuration.

• **Security headers:** Configure appropriate X-Content-Type-Options (commonly nosniff), Content Security Policy, Referrer Policy, and Strict-Transport-Security; test policies before deployment.

• **Credential handling:** Submit passwords in request bodies using POST over HTTPS rather than URLs. Avoid logging credentials and sensitive query parameters.

• **robots.txt:** Do not rely on it to protect sensitive paths; enforce authorization on sensitive resources.

• **Technology disclosure:** Review whether technology details need to be exposed, while prioritizing patching and secure configuration.

• **Authentication:** Consider rate limiting, monitoring, strong authentication, and suitable throttling. The finding indicates an interface was detected, not that brute force was demonstrated.

• **File uploads:** Use an allowlist, validate file content, generate server-side filenames, limit file size, and store uploads outside executable paths where feasible.

• **Security contact:** Publish a valid /.well-known/security.txt with an appropriate reporting contact and policy.

• **HTTP methods:** Review allowed methods and server configuration, and validate with response evidence before deciding whether changes are required.

These are defensive recommendations based on the report descriptions. They were not applied or retested during this exercise.

## 6. Method

• Opened the Google Gruyere training application and reviewed its safety warning.

• Started a temporary Gruyere instance and copied its instance-specific URL.

• Opened Pentest-Tools.com and selected New Scan → Website Scanner.

• Entered the temporary lab URL and verified Light scan, authentication disabled, custom scan time off, and same-domain redirects.

• Started the scan and waited for completion.

• Reviewed the summary, findings, performed tests, scan parameters, and scan statistics.

• Exported and retained the original scanner report as evidence.

## 7. Limitations and Interpretation

• This was a Light scan, not a comprehensive penetration test.

• Authentication was disabled, so authenticated areas were not assessed.

• SQLi, XSS, command injection, and XXE were not checked by this Light scan, as stated by the source report.

• Some findings require manual validation; password submission in URLs and technology fingerprinting were marked Unconfirmed.

• No exploitation, password guessing, denial-of-service testing, or remediation was performed.

• The lab instance is temporary; the exported report is the historical evidence for this run.

## 8. Conclusion

Exercise 2 was completed successfully. The Light scan finished in 17 seconds and reported Medium overall risk: 2 medium, 7 low, and 4 informational findings, with no critical or high findings. Key observations included missing cookie attributes, missing HTTP security headers, password inputs identified in GET forms, a login interface, a file-upload interface, and a missing security.txt file. The scan is useful initial evidence but cannot establish that the application is free of vulnerabilities.

## 9. Skills Demonstrated

Web vulnerability scanning • Interpreting scanner-generated risk ratings and evidence • HTTP security headers and cookie attributes • GET versus POST for sensitive inputs • Remediation reporting • Evidence preservation • Responsible use of an intentionally vulnerable training application.

## Portfolio Description

**Website Vulnerability Scanning — Exercise 2:** Performed a Light web vulnerability scan of a temporary Google Gruyere training instance using Pentest-Tools.com. Reviewed 40 scanner checks and documented 13 reported findings across medium, low, and informational categories. Analyzed cookie attributes, missing HTTP security headers, credential submission patterns, technology fingerprinting, and exposed application interfaces; recorded remediation guidance and scan limitations. Preserved the original scanner PDF and supporting screenshots. No exploitation was performed.

# 10. Evidence Screenshots

The accompanying evidence folder contains the scanner-exported PDF. The screenshots below document the lab setup, scan configuration, completion, findings, and scan statistics. The exported scanner PDF has been redacted to avoid publishing a cookie/session value in a public repository.

## 01. Lab overview



**Web Application Exploits and Defenses**

A Codelab by Bruce Leban, Mugdha Bendre, and Parisa Tabriz

**Want to beat the hackers at their own game?**

- Learn how hackers find security vulnerabilities!
- Learn how hackers exploit web applications!
- Learn how to stop them!

This codelab shows how web application vulnerabilities can be exploited and how to defend against these attacks. The best way to learn things is by doing, so you'll get a chance to do some real penetration testing, actually exploiting a real application. Specifically, you'll learn the following:

- How an application can be attacked using common web security vulnerabilities, like cross-site scripting vulnerabilities (XSS) and cross-site request forgery (XSRF).
- How to find, fix, and avoid these common vulnerabilities and other bugs that have a security impact, such as denial-of-service, information disclosure, or remote code execution.

To get the most out of this lab, you should have some familiarity with how a web application works (e.g., general knowledge of HTML, templates, cookies, AJAX, etc.).

**Gruyere**

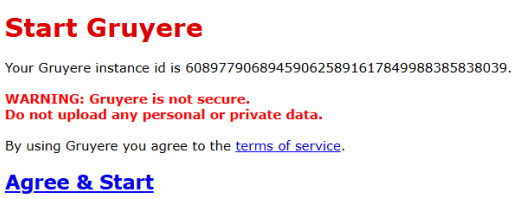
This codelab is built around **Gruyere** /gruːˈjɛər/ - a small, cheesy web application that allows its users to publish snippets of text and store assorted files. "Unfortunately," Gruyere has multiple security bugs ranging from cross-site scripting and cross-site request forgery, to information disclosure, denial of service, and remote code execution. The goal of this codelab is to guide you through discovering some of these bugs and learning ways to fix them both in Gruyere and in general.

The codelab is organized by types of vulnerabilities. In each section, you'll find a brief description of a vulnerability and a task to find an instance of that vulnerability in Gruyere. Your job is to play the role of a malicious hacker and find and exploit the security bugs. In this codelab, you'll use both black-box hacking and white-box hacking. In **black box hacking,** you try to find security bugs by experimenting with the application and manipulating input fields and URL parameters, trying to cause application errors, and looking at the HTTP requests and responses to guess server behavior. You do not have access to the source code, although understanding how to view source and being able to view http headers (as you can in Chrome or LiveHTTPHeaders for Firefox) is valuable. Using a web proxy like Burp or ZAP may be helpful in creating or modifying requests. In **white-box hacking,** you have access to the source code and can use automated or manual analysis to identify bugs. You can treat Gruyere as if it's open source: you can read through the source code to try to find bugs. Gruyere is written in Python, so some familiarity with Python can be helpful. However, the security vulnerabilities covered are not Python-specific and you can do most of the lab without even looking at the code. You can run a local instance of Gruyere to assist in your hacking: for example, you can create an administrator account on your local instance to learn how administrative features work and then apply that knowledge to the instance you want to hack. Security researchers use both hacking techniques, often in combination, in real life.



01-gruyere-lab-overview.png

## 02. Gruyere instance start warning



**Start Gruyere**

Your Gruyere instance id is 6089779068945906258916178499883858380 39.

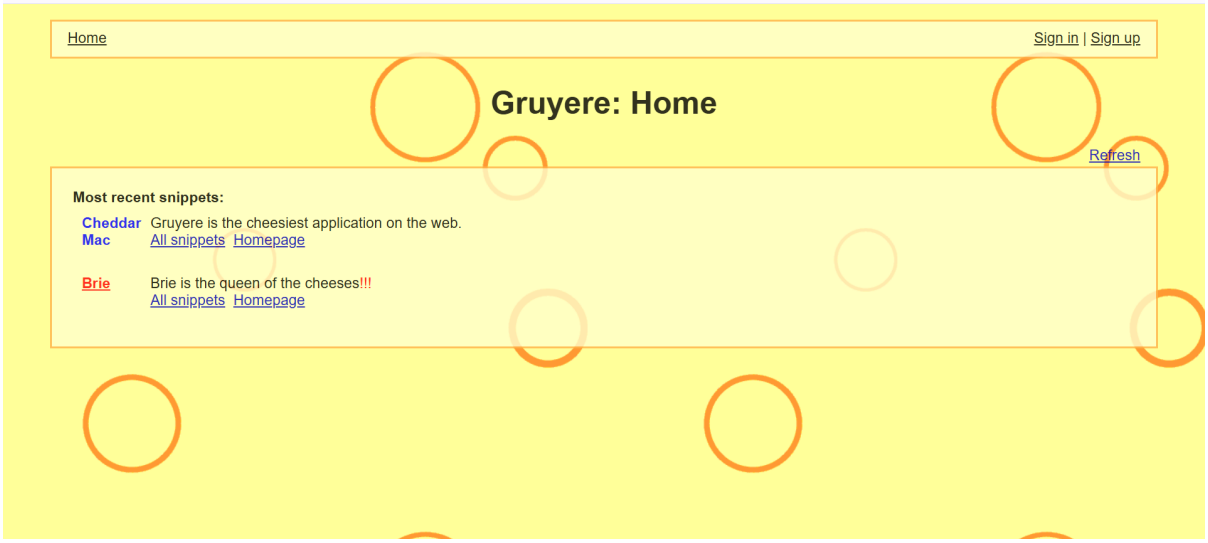
**WARNING: Gruyere is not secure.**
**Do not upload any personal or private data.**

By using Gruyere you agree to the terms of service.

**Agree & Start**

02-gruyere-instance-start.png

## 03. Gruyere homepage



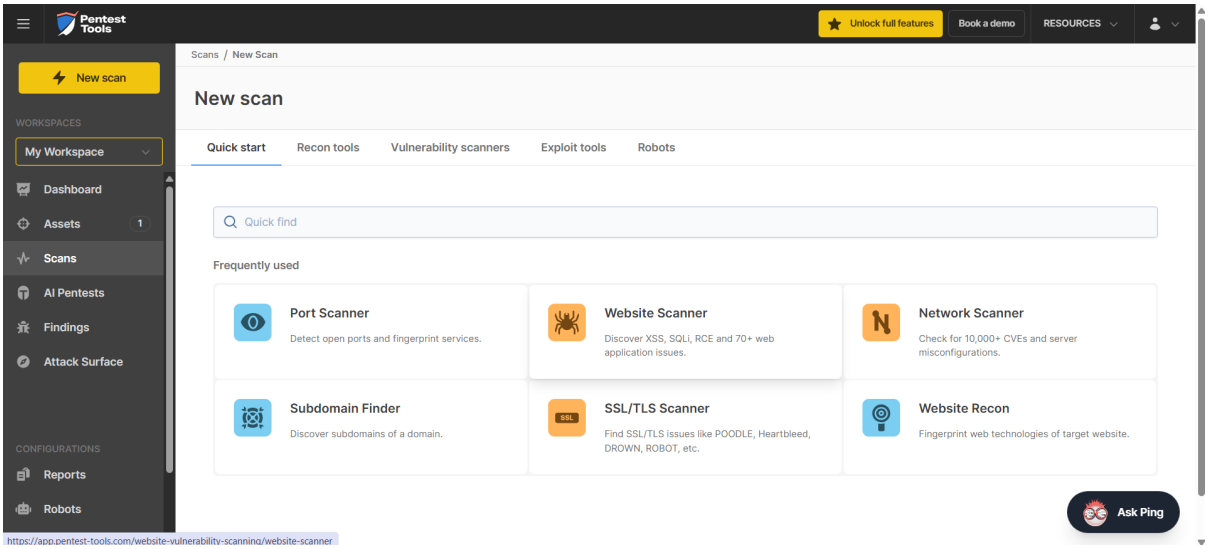

03-gruyere-homepage.png

## 04. Temporary instance URL



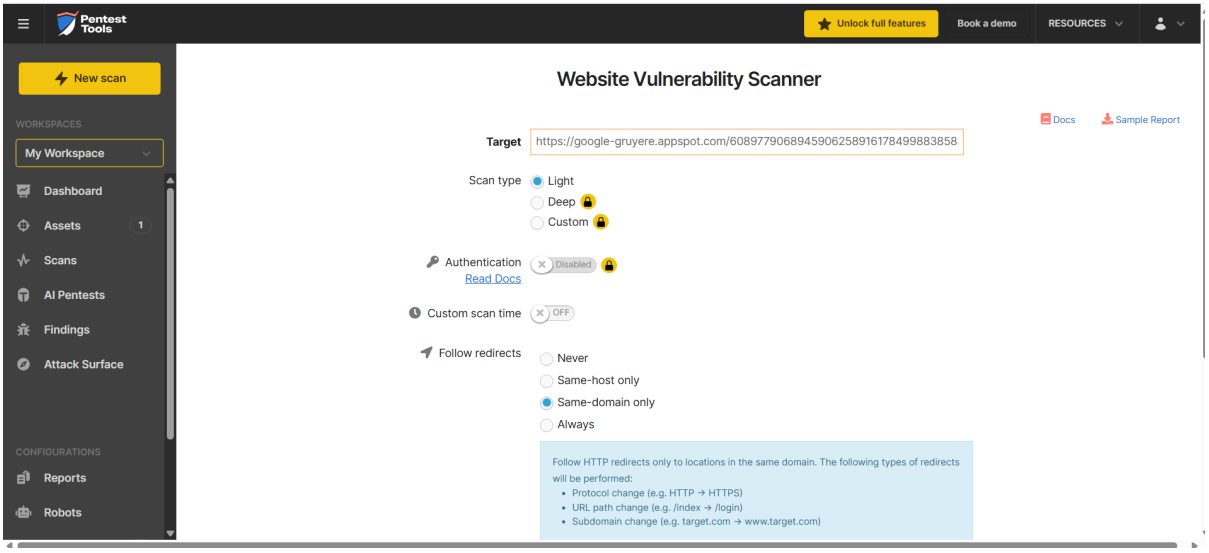

04-gruyere-instance-url.png

## 05. Scanner tool selection



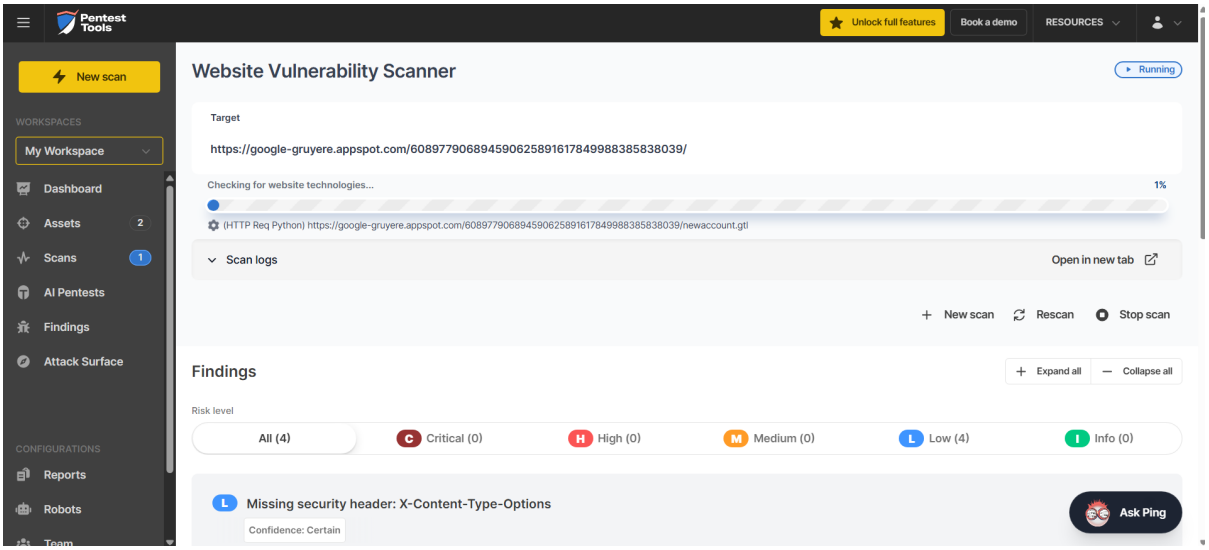

05-new-scan-tool-selection.png

## 06. Scan configuration



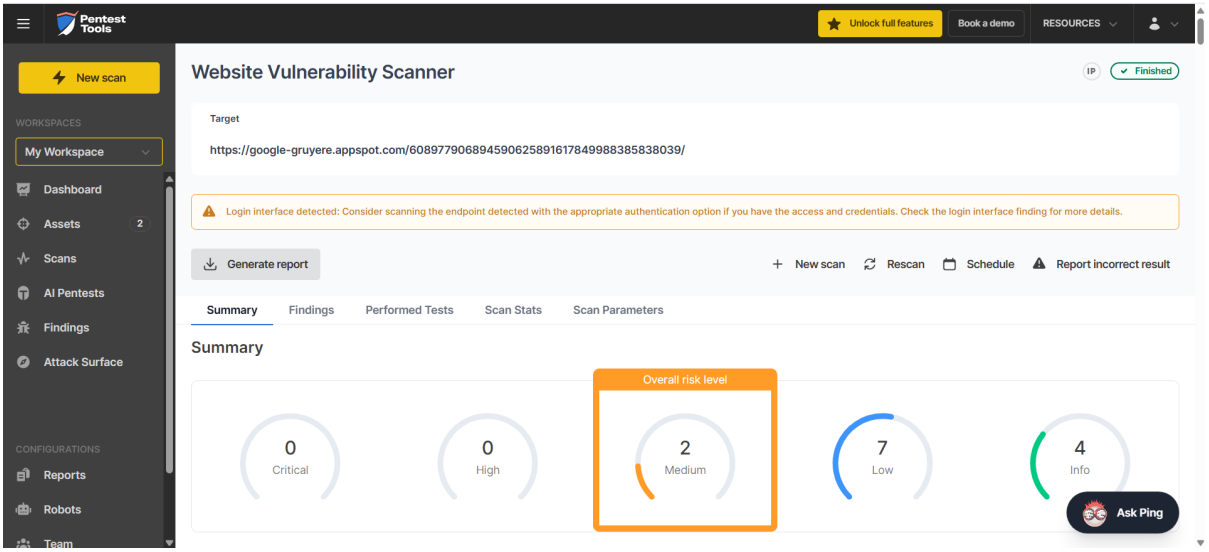

06-website-scan-configuration.png

## 07. Scan in progress



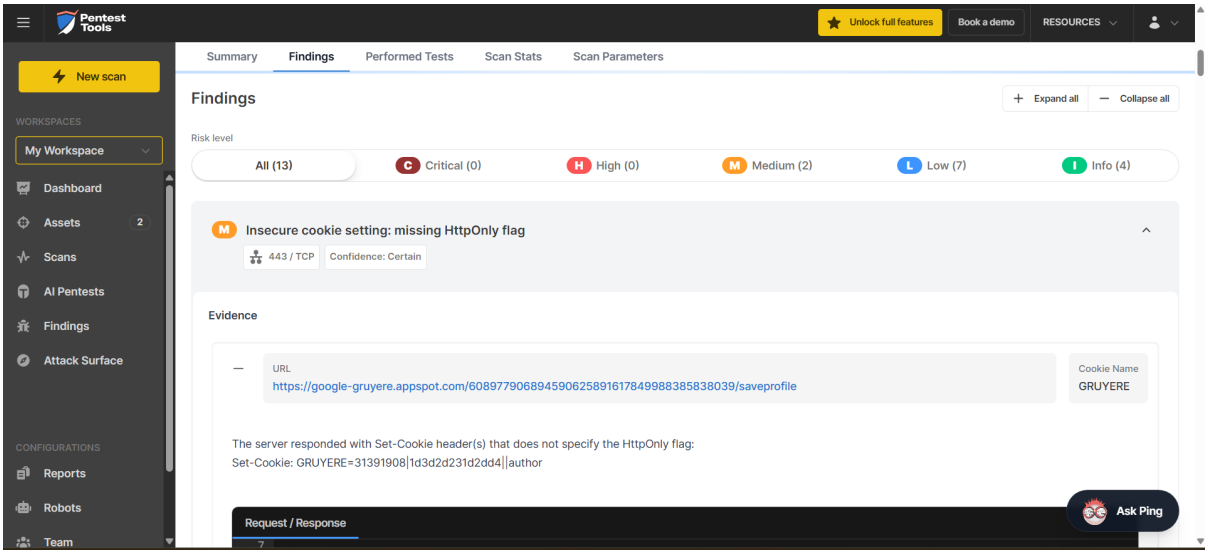

07-website-scan-in-progress.png

## 08. Completed summary



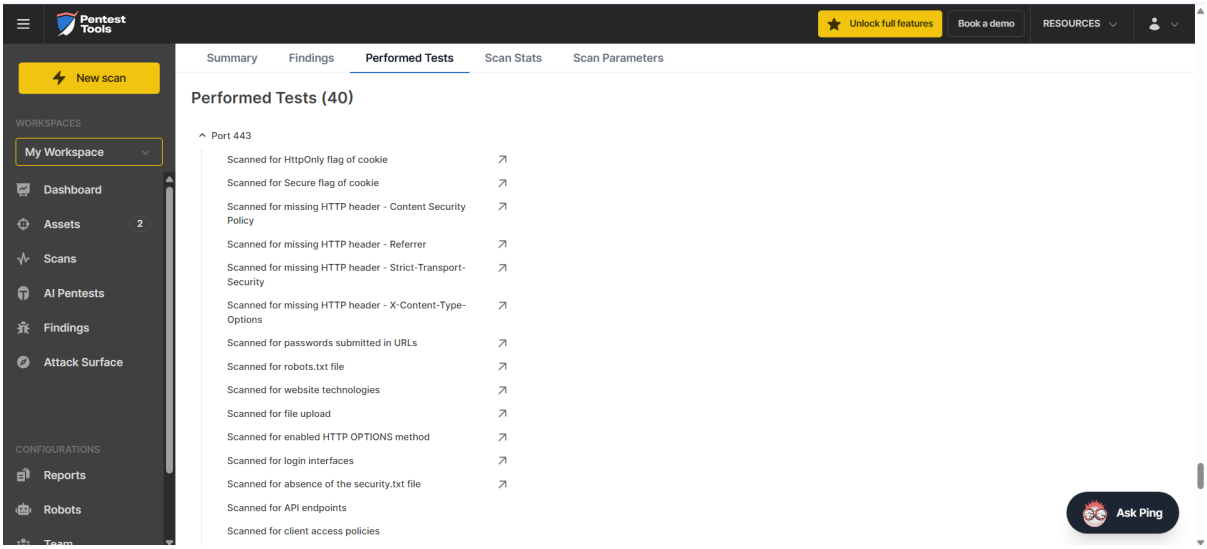

08-completed-scan-summary.png

## 09. Cookie finding



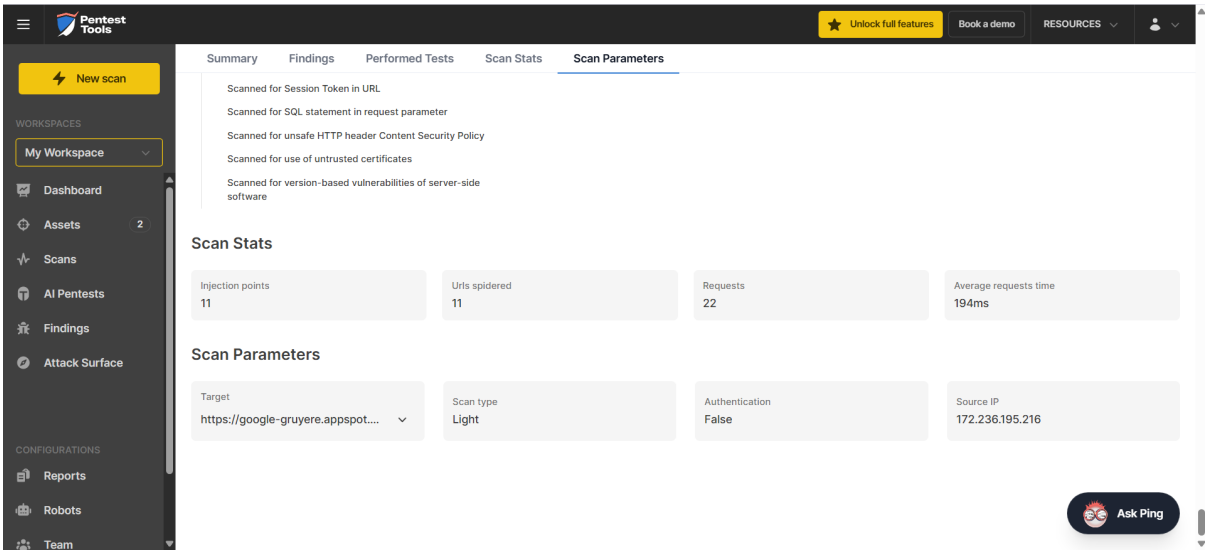

09-vulnerability-finding-cookie.png

## 10. Performed tests




10-performed-tests.png

## 11. Scan parameters and statistics




11-scan-parameters-and-stats.png